Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

TensorFlow Lite and ExecuTorch

Setting: both are frameworks for **on-device inference**: running a trained model directly on a phone, laptop, microcontroller or embedded device instead of calling a server. Benefits: low latency, offline use, privacy (data never leaves the device) and no server cost. Constraints: little memory, limited compute and battery, so models are converted to a compact format, usually quantized, and executed by a small runtime that can hand work to the device's GPU or NPU.

1. TensorFlow Lite (now LiteRT)

Google's runtime for on-device ML, born from the TensorFlow ecosystem and renamed **LiteRT** ("Lite Runtime") in 2024 as it opened to models from other frameworks (PyTorch and JAX via converters).



- **Converter:** turns the model into a `.tflite` file, a **FlatBuffer** that can be memory-mapped and read without parsing, with graph optimizations such as operator fusion.
- **Quantization:** float32 → float16 or int8 weights and activations, giving roughly 4× smaller models (int8) and faster integer math, using $x \approx s \cdot (q - z)$ with scale s and zero point z. Post-training or quantization-aware training.
- **Interpreter + delegates:** runs the graph on CPU (optimized kernels such as XNNPACK) and delegates supported parts to hardware accelerators: GPU, Android NPUs, Apple Core ML, Qualcomm Hexagon.
- **Platforms:** Android, iOS, Linux, web, and **microcontrollers** via TFLite Micro (models of a few hundred KB). **MediaPipe** builds ready-made solutions on top (face, hand, pose detection, on-device LLM inference).

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]          # enable quantization
open("model.tflite", "wb").write(converter.convert())
```

2. ExecuTorch

PyTorch's official solution for on-device inference, developed by Meta with the PyTorch community and successor of PyTorch Mobile. It keeps the model in the PyTorch ecosystem from training to the device, with no conversion to another framework's format.



- **Export:** `torch.export` captures the full model as a graph of standard PyTorch operators; it is lowered to the **Edge dialect**, a reduced operator set suitable for devices.
- **Backends (delegates):** parts of the graph are handed to optimized backends: **XNNPACK** (CPU), **Core ML** and **MPS** (Apple), **Vulkan** (GPU), **Qualcomm QNN**, **MediaTek** and **Arm** (NPUs).
- **Ahead-of-time:** planning (memory layout, operator selection) happens at export time, so the `.pte` program runs on a **small C++ runtime** with a very low footprint and no Python on the device.
- **Focus:** on-device LLMs and generative AI (Llama and other models on phones), used in production in Meta's apps; quantization through `torchao`.

```
ep  = torch.export.export(model, example_inputs)
pte = to_edge_transform_and_lower(ep, partitioner=[XnnpackPartitioner()]).to_executorch()
open("model.pte", "wb").write(pte.buffer)
```

Comparison:

	TensorFlow Lite / LiteRT	**ExecuTorch**
Ecosystem	Google, TensorFlow / Keras (now multi-framework)	Meta + PyTorch
Model format	`.tflite` (FlatBuffer)	`.pte` (ExecuTorch program)
Path to device	Convert to another format via converter	Stay in PyTorch: `torch.export` → lower
Acceleration	Delegates: GPU, NNAPI / NPUs, Core ML, Hexagon	Backends: XNNPACK, Core ML, MPS, Vulkan, QNN, Arm
MCUs	Yes, mature (TFLite Micro)	Supported, younger ecosystem
Maturity	Older, very widely deployed	Newer, growing fast, strong on LLMs
Choose if	Model in TF / Keras, MediaPipe, tiny MCUs	Model trained in PyTorch, on-device GenAI

In short: both turn a trained model into a compact, quantized file plus a lightweight runtime that uses the device's accelerators. The main choice is the **training framework**: TensorFlow / Keras points to LiteRT, PyTorch points to ExecuTorch.